



DESIGN AND IMPLEMENTATION OF A BUCKLE-ACTIVATED IOT SMART HELMET FOR AUTONOMOUS ACCIDENT DETECTION AND EMERGENCY ALERTING

A MAJOR PROJECT REPORT

Submitted by

SUGANTH S N [6176AC22UEC140]

THAMEEN ANSARI A [6176AC22UEC150]

VIDYADHEESHA M [6176AC22UEC162]

PANDURANGI

YESHWANTH R [6176AC22UEC170]

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

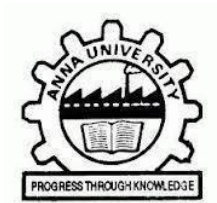
ELECTRONICS AND COMMUNICATION ENGINEERING

ADHIYAMAAN COLLEGE OF ENGINEERING [AUTONOMOUS]

Dr. M.G.R NAGAR, HOSUR - 635 130, TAMILNADU

ANNA UNIVERSITY:: CHENNAI 600 025

APRIL 2026



DESIGN AND IMPLEMENTATION OF A BUCKLE-ACTIVATED IOT SMART HELMET FOR AUTONOMOUS ACCIDENT DETECTION AND EMERGENCY ALERTING

A MAJOR PROJECT REPORT

Submitted by

SUGANTH S N [6176AC22UEC140]

THAMEEN ANSARI A [6176AC22UEC150]

VIDYADHEESHA M [6176AC22UEC162]

PANDURANGI

YESHWANTH R [6176AC22UEC170]

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

ELECTRONICS AND COMMUNICATION ENGINEERING

ADHIYAMAAN COLLEGE OF ENGINEERING [AUTONOMOUS]

Dr. M.G.R NAGAR, HOSUR - 635 130, TAMILNADU

ANNA UNIVERSITY:: CHENNAI 600 025

APRIL 2026

ANNA UNIVERSITY:: CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report titled “**DESIGN AND IMPLEMENTATION OF A BUCKLE-ACTIVATED IOT SMART HELMET FOR AUTONOMOUS ACCIDENT DETECTION AND EMERGENCY ALERTING**” is the bonafide work of **Suganth S N, Thameen Ansari A, Vidyadheesha M Pandurangi, Yeshwanth R** who carried out the research under my supervision.

SIGNATURE OF SUPERVISOR

Dr. S. SUMATHI., M.E., Ph.D.,
Professor & HOD of ECE Department,
Adhiyamaan College of Engineering
Dr. M. G. R Nagar
Krishnagiri (Dt.)
Hosur - 635109
Tamilnadu

SIGNATURE OF HOD

Dr. S. SUMATHI., M.E., Ph.D.,
Professor & HOD of ECE Department,
Adhiyamaan College of Engineering,
Dr. M. G. R Nagar
Krishnagiri (Dt.)
Hosur - 635109
Tamilnadu

Submitted for the Major Project Viva-Voce Examination held on _____ at

ADHIYAMAAN COLLEGE OF ENGINEERING, HOSUR - 635 130.

INTERNAL EXAMINER

EXTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	v
	ACKNOWLEDGEMENT	vi
	LIST OF FIGURES	vii
	LIST OF ABBREVIATIONS	viii
1	INTRODUCTION	1
2	LITERATURE SURVEY	2
3	EXISTING METHOD	4
4	PROPOSED METHOD	7
5	BLOCK DIAGRAM	9
6	WORKING	10
	6.1 CIRCUIT DIAGRAM	12
7	COMPONENTS SPECIFICATION	14
	7.1 ESP32 DEVELOPMENT BOARD (WROOM)	14
	7.2 GSM MODULE SIM800L	15
	7.3 GPS MODULE NEO-6M	16
	7.4 MPU6050 SENSOR	17
	7.5 HALL SENSOR	18
	7.6 BOOST CONVERTER	19
	7.7 BUCK CONVERTER	20
	7.8 P CHANNEL MOSFET	20
	7.9 TPU4056 CHARGING MODULE	21
	7.10 LITHIUM-ION BATTERY	22
8	PROGRAMMING IN EMBEDDED SYSTEM	23
9	RESULT AND DISCUSSION	25
10	CONCLUSION AND FUTURE ENHANCEMENT	27
	REFERENCE	28
	ANNEXURE I	29

ABSTRACT

Road accidents remain one of the leading causes of fatalities worldwide, often due to delayed emergency response and lack of real-time monitoring systems. Conventional helmets provide essential physical protection but lack intelligent features for accident detection and automated communication. To address this limitation, this project proposes an IoT-enabled Smart Helmet system designed to enhance rider safety through real-time accident detection and immediate emergency alerts.

The proposed system is built around the ESP32 microcontroller, which serves as the central processing and control unit. An MPU6050 accelerometer and gyroscope sensor continuously monitors motion parameters such as tilt angle and impact force to detect potential crash conditions. A Hall sensor ensures that the helmet is properly worn before system activation. In the event of a severe impact exceeding predefined threshold values, the GPS Neo-6M module retrieves real-time latitude and longitude coordinates, and the SIM800L GSM module automatically sends an emergency SMS alert containing the location details to predefined contacts.

By integrating motion sensing, location tracking, and GSM-based communication into a wearable safety device, the Smart Helmet provides a proactive and automated accident response mechanism. The system is cost-effective, scalable, and suitable for two-wheeler riders, delivery personnel, and commercial transportation sectors. This solution demonstrates how IoT technology can significantly improve road safety and reduce accident-related fatalities through intelligent monitoring and rapid emergency notification.

Keywords: IoT, Smart Helmet, ESP32, MPU6050, GPS Neo-6M, SIM800L, Accident Detection, Emergency Alert System.

ACKNOWLEDGEMENT

We wish to express my heartfelt gratitude to the department of Electronics and Communication Engineering, Adhiyamaan college of Engineering for the continued support, in technical expertise to excel to this technological world.

We take immense pleasure in thanking my honor. beloved and respected **PRINCIPAL Dr. R. RADHAKRISHNAN, M.E., Ph.D.**, for setting up an excellent atmosphere in this institution.

We wish to express the deepest gratitude to the Head of the Department. Electronics and Communication Engineering, **Dr. S. SUMATHI, M.E., Ph.D.**, who has been inspirational and supportive throughout my project.

We would like to thank our project supervisor, **Dr. S. SUMATHI, M.E., Ph.D.**, from the bottom of our heart, for her continuous guidance for the completion of the main project.

We also thank all the Teaching and Non-teaching staff who supported us in many ways for the completion of the project.

We are also deeply indebted to our family members and friends for their cooperation and suggestions towards the successful completion of the project

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
5.1	BLOCK DIAGRAM	9
6.1	CIRCUIT DIAGRAM	12
7.1	ESP32	14
7.2	GSM MODULE SIM800L	15
7.3	GPS MODULE NEO-6M	16
7.4	MPU6050 SENSOR	18
7.5	HALL SENSOR	19
7.6	BOOST CONVERTER	19
7.7	BUCK CONVERTER	20
7.8	P CHANNEL MOSFET	21
7.9	BATTERY CHARGING MODULE	21
7.10	LITHIUM ION BATTERY	22
9.1	CONTROLLING UNIT IN HELMET	25
9.2	POWER MANAGEMENT UNIT IN HELMET	25
9.3	HELMET OUT-LOOK	25
9.4	SMS ALERT TO USER & EMERGENCY CONTACT	26

LIST OF ABBREVIATIONS

IoT	Internet of Things
ESP32	Espressif Systems 32-bit Microcontroller
MPU6050	Motion Processing Unit (6-axis Accelerometer + Gyroscope)
GPS	Global Positioning System
GSM	Global System for Mobile Communication
SIM	Subscriber Identity Module
SMS	Short Message Service
UART	Universal Asynchronous Receiver and Transmitter
I ² C / I2C	Inter-Integrated Circuit
ADC	Analog-to-Digital Converter
GPIO	General Purpose Input/Output
PCB	Printed Circuit Board
DC	Direct Current

Li-ion	Lithium-Ion Battery
AT	Attention Command (used for GSM communication)
TX	Transmitter Line
RX	Receive Line
MCU	Microcontroller Unit
MEMS	Micro-Electro-Mechanical System (used in MPU6050 sensor)
GND	Ground
VCC	Voltage Common Collector (Power Supply Pin)
SMS Alert	One-Time Password Generation Algorithm
Auth	Authentication
ID	Identification Number
UI	User Interface
OSI	Open Systems Interconnection (Reference Model)

CHAPTER 1

INTRODUCTION

Road safety has become a critical global concern due to the increasing number of two-wheeler accidents and the delay in providing timely medical assistance to victims. While conventional helmets offer essential physical protection against head injuries, they lack intelligent monitoring and automated emergency response capabilities. In many accident scenarios, the absence of real-time communication and location tracking significantly increases the severity of consequences. Therefore, integrating modern sensing and communication technologies into protective gear is essential to enhance rider safety and reduce accident-related fatalities.

To address these challenges, this project proposes an IoT-based Smart Helmet system that combines motion sensing, location tracking, and GSM communication into a single integrated safety solution. The system is built around the ESP32 microcontroller. An MPU6050 accelerometer and gyroscope sensor continuously monitors motion parameters such as acceleration, orientation, and sudden impact forces to detect crash conditions. A Hall sensor ensures that the helmet is properly worn before system activation, reinforcing safety compliance. In the event of an accident, the GPS Neo-6M module acquires precise geographical coordinates, and the SIM800L GSM module automatically transmits an emergency alert message containing the rider's location to predefined contacts.

The proposed Smart Helmet system represents a proactive and intelligent approach to accident management. By automating crash detection and emergency communication, the system eliminates the need for manual intervention, ensuring rapid response even if the rider is unconscious. The integration of embedded systems, sensor technology, and wireless communication not only enhances operational reliability but also maintains cost-effectiveness and scalability. This innovative solution demonstrates how IoT-enabled wearable safety devices can significantly contribute to modern transportation safety frameworks and create a more responsive emergency support ecosystem.

CHAPTER 2

LITERATURE SURVEY

In[1], S. K. Bhoi and P. M. Khilar (2018), “*Vehicular Accident Detection and Reporting System Using Smartphones*”, published in **IEEE International Conference on Advanced Networks and Telecommunications Systems** proposed a smartphone-based accident detection system using cloud communication. While it provides effective location tracking, it depends heavily on smartphone availability and user interaction, which may not be reliable during severe accidents.

In[2], D. Joshi and S. Mehta (2018), “*Accident Detection System Using Sensors*”, published in **International Journal of Engineering Research & Technology (IJERT)**. This system uses ultrasonic and vibration sensors to detect collisions. Although effective for vehicle-based detection, it lacks rider-specific monitoring capability.

In[3], T. Patel and D. Shah (2019), “*Real-Time Accident Detection Using Accelerometer and GSM*”, published in **International Journal of Innovative Technology and Exploring Engineering (IJITEE)**. The study demonstrates that threshold-based motion analysis can detect accidents effectively, but the absence of a verification mechanism may lead to false alerts.

In[4], R. Kumar and S. Raj (2020), “*IoT-Based Smart Helmet for Accident Detection*”, published in **IEEE Xplore Digital Library**. The system integrates motion sensors and GSM communication for real-time accident detection, improving emergency response time.

In[5], K. S. Reddy and V. Lakshmi (2020), “*Smart Helmet with Alcohol Detection and Accident Notification*”, published in **International Journal of Engineering and Advanced Technology (IJEAT)**. The work combines preventive and post-accident safety mechanisms in a wearable device.

In[6], J. Lee et al. (2020), “*Wearable Safety Device Using MEMS Sensors*”, published in **IEEE Sensors Journal**. The system uses MEMS sensors with Bluetooth communication but is limited to short-range operation., K. S. Reddy and V. Lakshmi (2020) introduced a smart helmet with alcohol detection and accident notification features, enhancing preventive safety.

In[7], M. S. Hossain and G. Muhammad (2021), “*IoT-Based Intelligent Helmet System*”, published in **IEEE Internet of Things Journal**. The research confirms that sensor fusion improves accident detection accuracy in wearable systems., M. S. Hossain and G. Muhammad (2021) developed an IoT helmet using MEMS sensors with sensor fusion to improve accident detection accuracy.

In[8], Y. Zhang et al. (2021), “*MEMS-Based Sensor Systems for Intelligent Transportation*”, published in **Sensors Journal (MDPI)**. The study highlights improved reliability through accelerometer and gyroscope integration.

In[9], K. Ahmed and M. Rahman (2021), “*GSM and GPS-Based Accident Alert System*”, published in **International Journal of Computer Science Engineering**. Provides reliable alert transmission but lacks false alarm verification.

In[10], M. Khan et al. (2021), “*IoT-Based Transportation Safety System*”, published in **International Journal of Smart Technology**. The system ensures real-time tracking but lacks wearable integration.

In[11], S. N. Shinde and P. Patil (2024), “*Two-Wheeler Accident Detection Using GSM and GPS*”, published in **International Journal of Advanced Research in Engineering and Technology (IJARET)**. Provides real-time alerts but lacks a confirmation mechanism.

In[12], V. Rao and S. Prasad (2024), “*Accident Detection System with SMS Verification*”, published in **IEEE Conference on Smart Systems**. Introduces SMS-based verification to reduce false alerts and improve reliability.

CHAPTER 3

EXISTING METHOD

Before the development of IoT-enabled smart helmets, several conventional and semi-automated safety approaches were implemented to reduce accident severity and improve emergency response. However, these systems lacked full integration, automation, and wearable intelligence.

A. Conventional Safety Helmets

Traditional helmets are designed primarily for impact protection using hard outer shells and shock-absorbing inner materials. Their sole purpose is to minimize head injuries during collisions.

Working Principle:

Mechanical shock absorption reduces the force transmitted to the rider's skull during impact

Limitations:

- No accident detection capability.
- No emergency communication mechanism.
- Cannot transmit rider location.
- No monitoring of riding conditions or abnormal motion.

Although effective for physical protection, conventional helmets provide no intelligent safety assistance.

B. Vehicle-Mounted GPS Tracking Systems

Several two-wheelers and commercial vehicles are equipped with standalone GPS tracking devices for fleet monitoring and theft prevention.

Working Principle:

GPS modules continuously track vehicle location and transmit data to a monitoring centre via GSM networks.

Limitations:

- Does not detect accidents automatically.
- Requires manual monitoring.
- Cannot determine crash severity.
- Location tracking alone does not ensure emergency response.

These systems focus on tracking rather than accident management.

C. Manual GSM Emergency Alert Systems

Some early safety systems used GSM modules to send emergency SMS alerts when triggered manually.

Working Principle:

A panic switch or button sends an SMS to predefined contacts via GSM.

Limitations:

- Requires manual activation.
- Ineffective if rider is unconscious.
- No motion analysis to confirm actual accidents.
- High chances of false alerts.

These systems depend entirely on rider response capability.

D. Basic Accelerometer-Based Accident Detection Units

Some research prototypes and vehicle safety devices used standalone accelerometers to detect sudden impacts.

Working Principle:

Sudden acceleration beyond a threshold value indicates a potential crash.

Limitations:

- Often generates false positives due to road bumps.
- No GPS integration for location tracking.
- No automatic GSM alert transmission.
- Not integrated into wearable protective equipment.

These systems lacked comprehensive integration and real-time communication.

Summary of Gaps in Existing Methods

Existing methodologies either focus on physical protection, location tracking, or manual alert mechanisms independently. None provide a fully integrated wearable system that combines:

- Real-time motion sensing
- Automatic accident detection
- Immediate GPS-based location tracking
- Autonomous GSM emergency alert transmission
- Helmet-wearing verification

These limitations highlight the necessity for an IoT-enabled Smart Helmet system that integrates MPU6050 motion sensing, GPS Neo-6M tracking, SIM800L GSM communication, and ESP32-based intelligent processing into a single automated safety framework.

CHAPTER 4

PROPOSED METHOD

The proposed Smart Helmet is an IoT-enabled intelligent safety system that integrates motion sensing, GPS tracking, GSM communication, and helmet-wearing verification into a compact wearable device. The ESP32 microcontroller acts as the central processing and decision-making unit, managing sensor acquisition, accident detection algorithms, verification logic, and communication protocols in real time.

1. Helmet-Wearing Verification:

A Hall sensor is embedded within the helmet strap to ensure proper fastening. The system activates only when the helmet is securely worn, enforcing rider compliance and preventing unnecessary operation when the helmet is not in use.

2. Accident Detection Using MPU6050:

The MPU6050 sensor continuously monitors acceleration, angular velocity, and tilt orientation. The ESP32 analyses this data against predefined threshold values.

An accident condition is identified based on:

- Sudden high acceleration (impact force)
- Abnormal tilt angle beyond safe limits
- Rapid orientation changes within a short duration

To reduce false triggers caused by road irregularities, a short validation delay is applied before proceeding to the verification stage.

3. Accident Verification Mechanism

Before notifying emergency contacts, the system implements a user confirmation process to prevent false alarms.

✓ Once a potential accident is detected, an SMS alert is first sent to the rider's registered mobile number.

✓ The message prompts the rider to reply with "YES" if an accident has occurred or "NO" if it is a false detection.

Response Handling Logic:

❖ If the rider replies "YES", the system immediately sends an emergency SMS to predefined emergency contacts along with the GPS location.

- ❖ If the rider replies “**NO**”, the system considers it a false alarm and resumes normal monitoring.
- ❖ If no reply is received within 5 minutes, the system automatically assumes that the rider may be unconscious and proceeds to send the emergency alert to contacts.

4. GPS Location Tracking

Upon confirmation of an accident (either by user reply or timeout), the GPS Neo-6M module retrieves real-time latitude and longitude coordinates via UART communication. The system ensures valid satellite lock before proceeding.

5. GSM-Based Emergency Alert Transmission

The ESP32 communicates with the SIM800L GSM module using AT commands to send the emergency SMS.

The alert message includes:

- Accident notification statement
- Exact latitude and longitude coordinates
- Google Maps location link for easy navigation

6. System Automation and Reliability

The entire workflow—from motion detection to verification and emergency alert transmission—is fully autonomous. The ESP32 manages power optimization, sensor polling, message parsing, and timeout control while operating on a compact Li-ion battery supply.

By integrating motion sensing (MPU6050), GPS tracking (Neo-6M), GSM communication (SIM800L), and intelligent verification logic into a unified embedded framework, the proposed Smart Helmet provides a reliable, real-time, and fault-tolerant accident detection and emergency response system tailored for two-wheeler riders.

CHAPTER 5

BLOCK DIAGRAM

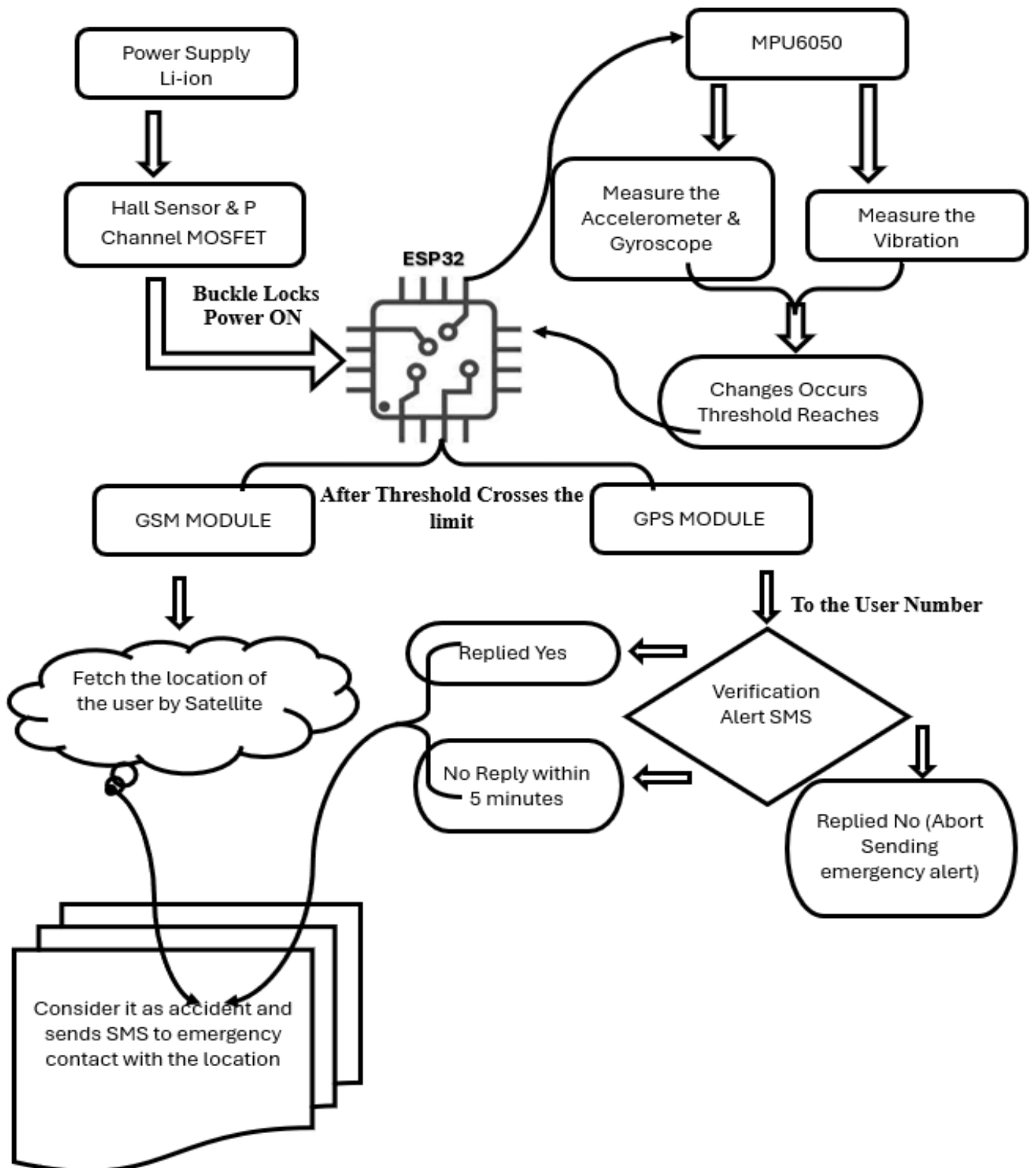


Figure 5.1 Block Diagram

CHAPTER 6

WORKING

- ✓ When power is supplied from the Li-ion battery, initially only the Hall sensor circuit is energized, while the ESP32, MPU6050, GPS Neo-6M, and SIM800L modules remain powered OFF to conserve energy.
- ✓ When the helmet strap is properly fastened, the magnet placed near the Hall sensor produces a magnetic field. The Hall sensor detects this field and generates a trigger signal.
- ✓ This trigger signal activates the PNP transistor, which acts as a high-side power switch and supplies power to the entire system, including the ESP32 and all connected modules.
- ✓ Once powered, the ESP32 initializes the MPU6050 through I2C communication and establishes UART communication with the GPS Neo-6M and SIM800L GSM module. The system then enters continuous monitoring mode.
- ✓ During riding, the MPU6050 continuously measures acceleration, angular velocity, and tilt orientation. The ESP32 processes these sensor values in real time and compares them with predefined threshold limits.
- ✓ If the sensor readings indicate sudden high acceleration, abnormal tilt angle, or rapid orientation change beyond safe limits, the system identifies it as a potential accident condition.
- ✓ To avoid false detection caused by road bumps or sudden braking, the ESP32 verifies whether the abnormal values persist for a short duration before confirming the accident.
- ✓ After confirmation of a potential accident, the system first sends an SMS to the rider's registered mobile number through the SIM800L module, requesting verification. The rider is asked to reply "YES" if the accident is real or "NO" if it is a

false alarm.

- ✓ The system waits for a response for up to five minutes. If the rider replies “NO,” the system considers it a false detection and resumes normal monitoring.
- ✓ If the rider replies “YES,” the system immediately proceeds to send an emergency alert.
- ✓ If “NO” response is received within five minutes, the system assumes that the rider may be unconscious and automatically treats the situation as a confirmed accident.
- ✓ The ESP32 then activates the GPS Neo-6M module to obtain accurate latitude and longitude coordinates after ensuring valid satellite connectivity.
- ✓ Using the SIM800L GSM module, the system sends an emergency SMS to predefined emergency contacts. The message includes an accident alert statement along with the GPS coordinates and a Google Maps link for easy access to the rider’s location.
- ✓ After sending the emergency notification, the system continues to monitor motion data until the helmet strap is unfastened.
- ✓ When the helmet strap is removed, the Hall sensor no longer detects the magnetic field, the PNP transistor turns OFF, and power to the entire system is cut off automatically, returning the system to low-power standby mode.

6.1 Circuit Diagram

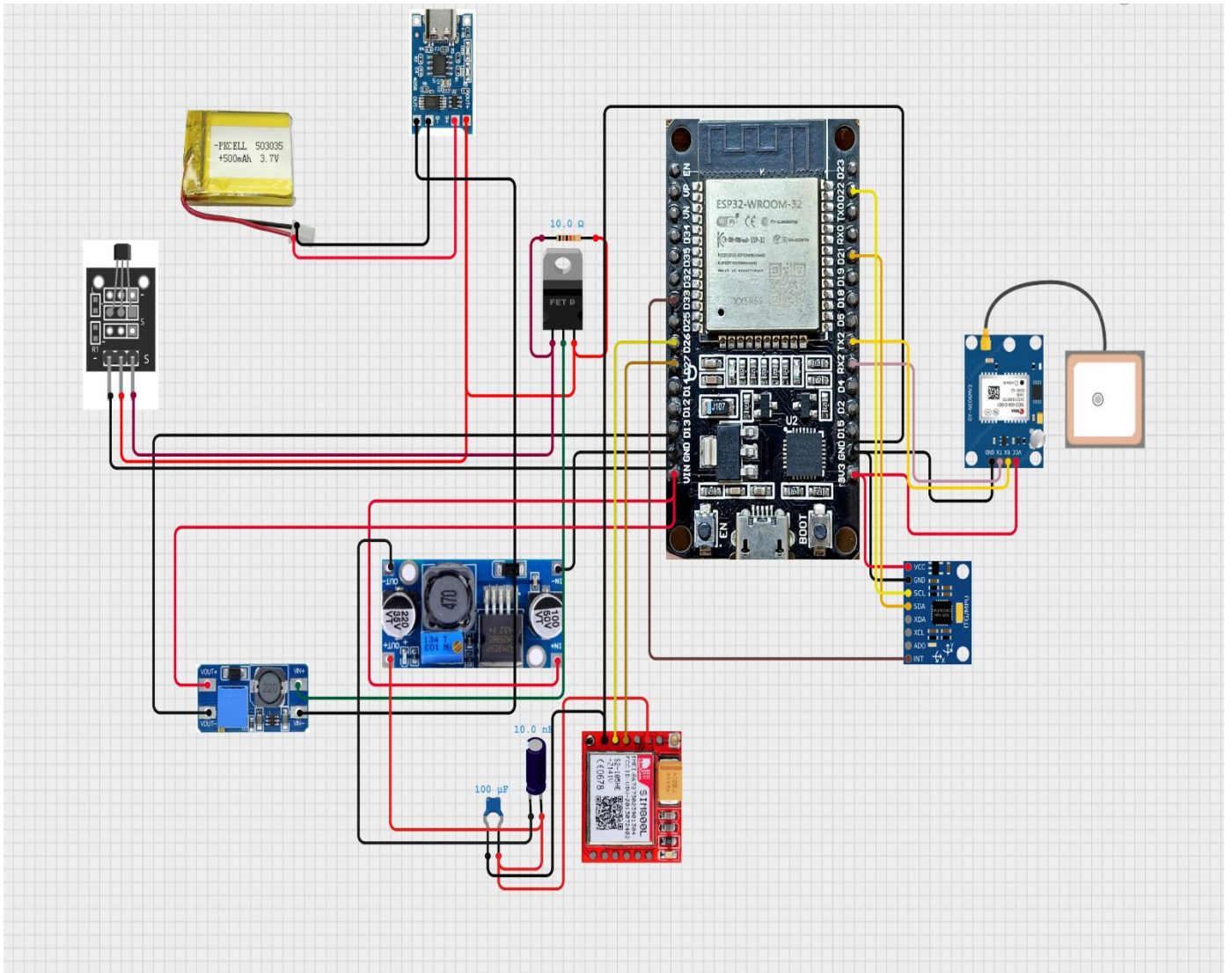


Figure 6.1 Circuit Diagram

S.NO	Module	Module Pin	Connect
1	Battery	+	TPU4056 B+
2	Battery	-	TPU4056 B-
3	TPU4056	OUT +	Hall Sensor VCC & MOSFET Source
4	TPU4056	OUT -	Boost VIN -
5	Hall Sensor	GND	Common GND
6	Hall Sensor	Signal	Gate
7	MOSFET	Gate	Source (10kohm R)
8	MOSFET	Drain	Boost VIN+
9	Boost	VOUT+	ESP32 VIN
10	Boost	VOUT-	ESP32 GND
11	ESP32	VIN	Buck IN+

12	ESP32	GND	Buck IN-
13	Buck	OUT+	GSM VCC (1000mF Electrolyte & 100mFcermaic Capacitor)
14	Buck	OUT-	GSM GND (1000mF Electrolyte & 100mFcermaic Capacitor) – Common GND
15	ESP32	3.3V	GPS VCC & MPU6050
16	ESP32	GND	GPS GND & MPU6050
17	ESP32	RX2	GPS TX
18	ESP32	TX2	GPS RX
19	ESP32	Piu 26	GSM TX
20	ESP32	Pin 27	GSM RX
21	ESP32	Pin 22	MPU6050 SCL
22	ESP32	Pin 21	MPU6050 SDA
23	ESP32	Pin 33	MPU6050 INT

CHAPTER 7

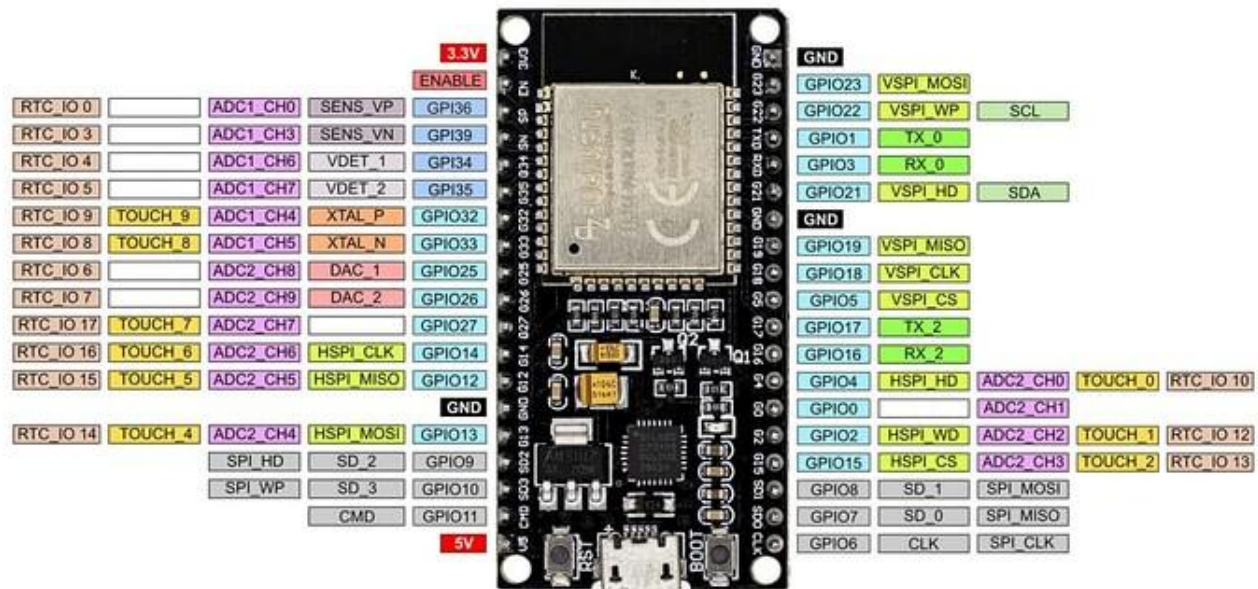
COMPONENTS SPECIFICATION

7.1 ESP32 Development Board (WROOM)

The ESP32 is a high-performance, low-power 32-bit microcontroller developed by Espressif Systems. It features a dual-core Xtensa LX6 processor, integrated Wi-Fi and Bluetooth capabilities, multiple GPIO pins, ADC, DAC, I2C, SPI, and UART communication interfaces.

In this project, the ESP32 functions as the central processing and control unit. It performs the following operations:

Figure 7.1 ESP32



- ✓ Reads motion data from the MPU6050 sensor via I2C protocol.
- ✓ Executes the accident detection algorithm based on threshold values.
- ✓ Controls the accident verification logic and 5-minute timeout mechanism.
- ✓ Communicates with the SIM800L GSM module using UART for SMS transmission and reception.
- ✓ Retrieves real-time location data from the GPS Neo-6M module.
- ✓ Manages overall system power and decision-making operations.

The ESP32 ensures real-time processing, fast communication, and efficient power consumption, making it suitable for wearable IoT applications

7.2 GSM MODULE SIM800L

The SIM800L is a quad-band GSM/GPRS communication module capable of sending SMS, making calls, and transmitting data over cellular networks.

Working Principle:

The module communicates with the ESP32 using AT commands via UART communication. It requires a SIM card to access the mobile network.

In this project, the SIM800L performs two major functions:

- ✓ Sends an initial verification SMS to the rider after accident detection.
- ✓ Receives the rider's reply ("YES" or "NO").
- ✓ Sends emergency alert SMS to predefined contacts if accident is confirmed or no response is received within 5 minutes.

The emergency SMS includes a Google Maps link containing GPS coordinates for easy location access.

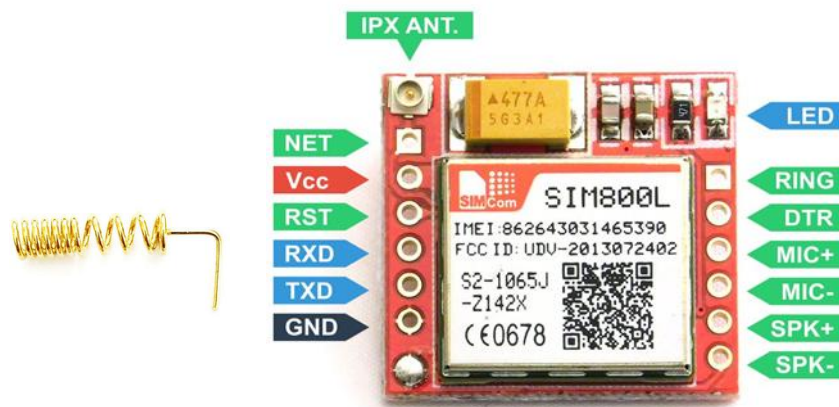


Figure 7.2 GSM MODULE SIM800L

7.3 GPS MODULE NEO-6M

A Neo-6M is a satellite-based GPS receiver module that provides accurate geographical positioning.

Working Principle:

The module receives signals from multiple GPS satellites and calculates latitude and longitude coordinates using trilateration. It communicates with the ESP32 through UART.

In this project, when an accident is confirmed, the ESP32 activates the GPS module to acquire valid location data. The coordinates are embedded into the emergency message in the form of:

- ✓ Latitude
- ✓ Longitude
- ✓ Google Maps URL

This ensures precise location tracking for emergency responders.

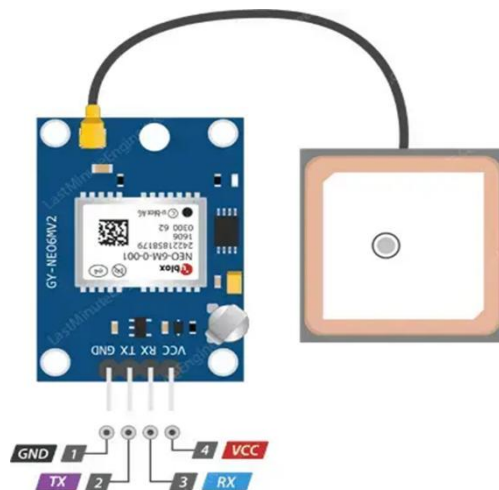


Figure 7.3 GPS MODULE NEO-6M

7.4 MPU6050 SENSOR

The MPU6050 is a 6-axis MEMS (Micro-Electro-Mechanical System) motion tracking device that integrates a 3-axis accelerometer and a 3-axis gyroscope within a single IC. It also contains an onboard Digital Motion Processor (DMP), which helps in processing motion data efficiently.

In this Smart Helmet project, the MPU6050 plays a critical role in detecting sudden impacts, abnormal tilt angles, and vibration patterns that indicate potential accident conditions.

Working Principle:

The MPU6050 is a motion sensor that contains a 3-axis accelerometer and a 3-axis gyroscope. The accelerometer measures linear acceleration along X, Y, and Z axes, while the gyroscope measures rotational motion. In this project, the accelerometer is mainly used to detect vibration and sudden impact.

How It Detects Vibration:

- ✓ The accelerometer continuously measures acceleration values in all three axes.
- ✓ During normal conditions, the acceleration values change slowly and remain stable.
- ✓ When vibration occurs, the acceleration values fluctuate rapidly within a short time.
- ✓ The ESP32 reads these values and checks the difference between consecutive readings.
- ✓ If rapid and repeated small changes are detected above a predefined vibration threshold, it identifies it as vibration.
- ✓ If the acceleration suddenly exceeds a high threshold (for example $>3g$), it is considered a potential accident instead of normal vibration.

Configuration Using Interrupt Pin:

The MPU6050 has an INT (Interrupt) pin that signals the microcontroller when specific motion events occur.

- ✓ First, the sensor is configured through I2C communication.

- ✓ The motion threshold register is set to define how much acceleration change should trigger an interrupt.
- ✓ The motion duration register is configured to define how long the motion must persist before triggering.
- ✓ The interrupt enable register is activated to allow motion detection interrupt generation.
- ✓ The INT pin of MPU6050 is connected to a GPIO pin of ESP32.
- ✓ When vibration or motion exceeds the configured threshold, the MPU6050 automatically pulls the INT pin HIGH.
- ✓ The ESP32 detects this interrupt signal and immediately reads sensor data for further verification.

Using the interrupt pin improves efficiency because the ESP32 does not need to continuously poll the sensor; instead, it responds only when motion activity is detected.

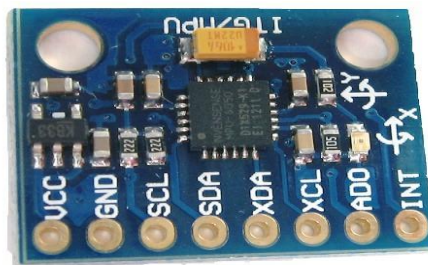


Figure 7.4 MPU6050 SENSOR

7.5 HALL SENSOR

The Hall sensor operates based on the Hall Effect principle, where a voltage is generated when a magnetic field is detected perpendicular to current flow.

In this project:

- ✓ A magnet is placed in the helmet strap.
- ✓ When the strap is fastened, the Hall sensor detects the magnetic

field.

- ✓ It sends a trigger signal to activate the PNP transistor.

This mechanism ensures that the system powers ON only when the helmet is properly worn.

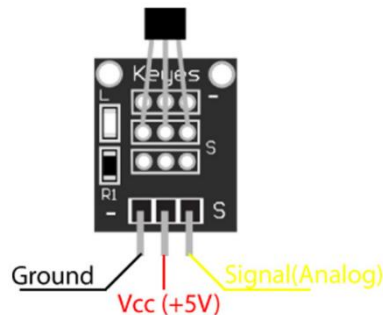


Figure 7.5 HALL SENSOR

7.6 BOOST CONVETER

A boost converter is a DC-DC step-up voltage regulator that increases a lower input voltage to a higher output voltage. It operates using switching components such as an inductor, diode, capacitor, and switching transistor to store energy and release it at a higher voltage level. The output voltage is regulated by controlling the switching frequency.

In this Smart Helmet project, the boost converter is used to increase the 3.7V output from the Li-ion battery to a higher stable voltage (such as 5V) required by modules like SIM800L or other components that demand higher operating voltage. It ensures stable performance and prevents voltage drop during GSM transmission, which typically requires higher current.

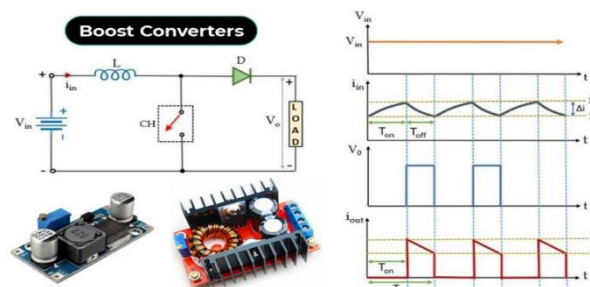


Figure 7.6 BOOST CONVERTER

7.7 BUCK CONVETER

A buck converter is a DC-DC step-down voltage regulator that reduces a higher input voltage to a lower, stable output voltage. It works by rapidly switching a transistor on and off and filtering the output through an inductor and capacitor to maintain a steady reduced voltage.

In this project, the buck converter is used when a stable lower voltage (such as 3.3V) is required for components like ESP32 or sensors. It ensures safe voltage regulation and protects sensitive components from overvoltage damage while maintaining high efficiency and low heat generation.

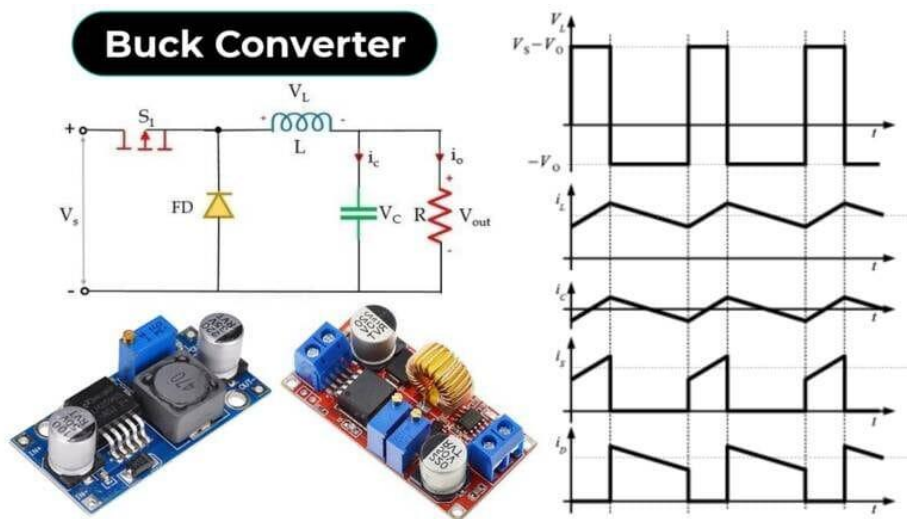


Figure 7.7 BUCK CONVERTER

7.8 P CHANNEL MOSFET

The FQPF27P06 is a P-Channel Power MOSFET designed for high-current and high-efficiency switching applications. It operates as a voltage-controlled device, where the flow of current between the source and drain terminals is controlled by the voltage applied to the gate terminal. Unlike bipolar transistors, MOSFETs require very low gate current, making them highly efficient for power switching circuits. The FQPF27P06 can handle high current loads and typically operates with a maximum drain-source voltage of around -60V and a high current rating, making it suitable for battery-operated systems.

In this Smart Helmet project, the FQPF27P06 P-Channel MOSFET is used as a high-side power switch to control the main power supply to the ESP32 and other modules. Initially, only the Hall sensor is powered. When the helmet strap is fastened, the Hall sensor output triggers the MOSFET gate, allowing current to flow from the Li-ion battery to the entire system. When the strap is removed, the MOSFET turns OFF, cutting power to all components except the Hall sensor. This design significantly improves power efficiency, reduces unnecessary battery drain, and ensures that the system operates only when the helmet is properly worn.

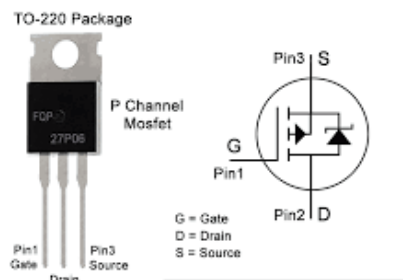


Figure 7.8 P CHANNEL MOSFET

7.9 TPU4056 CHARGING MODULE

The TP4056 is a lithium-ion battery charging controller module designed specifically for single-cell Li-ion batteries. It provides constant current and constant voltage charging with built-in protection features such as overcharge protection and charge termination.

In this Smart Helmet system, the TP4056 module is used to safely recharge the 3.7V Li-ion battery through a USB power source. It regulates charging current, prevents overcharging, and ensures battery safety, thereby extending battery life and improving overall system reliability.

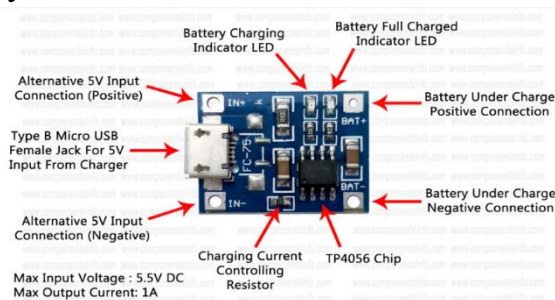


Figure 7.9 P TPU6050 CHARGING MODULE

7.10 LITHIUM-ION BATTERY

A Lithium-Ion battery is a rechargeable energy storage device known for high energy density, lightweight structure, and long cycle life. It typically provides a nominal voltage of 3.7V per cell and is widely used in portable electronic devices.

In this project, the Li-ion battery serves as the primary power source for the entire Smart Helmet system. It supplies power to the Hall sensor, ESP32, sensors, and communication modules. Its compact size and rechargeable capability make it ideal for wearable applications, ensuring portability and uninterrupted operation.



Figure 7.10 LITHIUM-ION BATTERY

CHAPTER 8

PROGRAMMING IN EMBEDDED SYSTEMS

As mentioned earlier, Embedded Systems consists of both Hardware and Software. If we consider a simple Embedded System, the main Hardware Module is the Processor. The Processor is the heart of the Embedded System, and it can be anything like a Microprocessor, Microcontroller, DSP, CPLD (Complex Programmable Logic Device) and FPGA (Field Programmable Gated Array).

All these devices have one thing in common: they are programmable, i.e. we can write a program (which is the software part of the Embedded System) to define how the device works.

Embedded Software or Program allow Hardware to monitor external events (Inputs) and control external devices (Outputs) accordingly. During this process, the program for an Embedded System may have to directly manipulate the internal architecture of the Embedded Hardware (usually the processor) such as Timers, Serial Communications Interface, Interrupt Handling, and I/O Ports etc.

From the above statement, it is clear that the Software part of an Embedded System is equally important to the Hardware part. There is no point in having advanced Hardware Components with poorly written programs (Software).

There are many programming languages that are used for Embedded Systems like Assembly (low-level Programming Language), C, C++, JAVA (high-level programming languages), Visual Basic, JAVA Script (Application-level Programming Languages), etc. In the process of making a better embedded system, the programming of the system plays a vital role, and hence, the selection of the Programming Language is very important.

EMBEDDED C

Embedded C is one of the most popular and most used Programming Languages in the development of Embedded Systems. So, in this article, we will see some of the Basics of Embedded C Program and the Programming Structure of Embedded C.

Embedded C is perhaps the most popular language among Embedded Programmers for programming Embedded Systems. There are many popular programming languages like Assembly, BASIC, C++ etc. that are often used for developing Embedded Systems, but Embedded C remains popular due to its efficiency, less development time and portability.

Before digging into the basics of Embedded C Program, we will first take a look at what an Embedded System is and the importance of Programming Language in Embedded Systems.

CHAPTER 9

RESULTS AND DISCUSSIONS

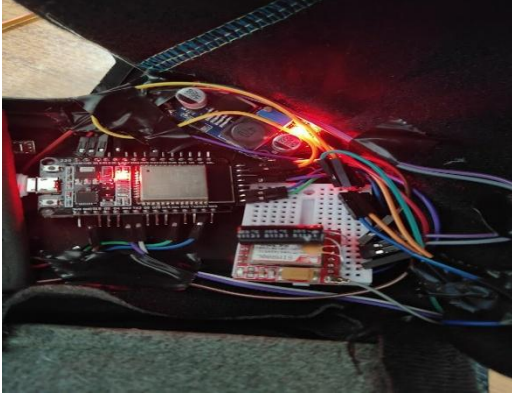


Figure 9.1: Controlling Unit in Helmet



Figure 9.2: Power Management Unit in Helmet



REAR VIEW



SIDE VIEW



FRONT VIEW

Figure 9.3: Helmet Out-Look

The proposed IoT-based Smart Helmet system was tested under various simulated riding and accident conditions to evaluate its performance, reliability, and response time. Experimental validation was carried out by generating controlled impact scenarios and monitoring acceleration values using the MPU6050 sensor. During normal riding conditions, the resultant acceleration remained close to 1g with minor fluctuations due to road vibrations. However, during simulated crash conditions, sudden spikes exceeding the predefined threshold (3g–4g) were consistently observed, enabling accurate accident detection.

Power Management and System Reliability

Power consumption analysis indicated that the Hall sensor-controlled MOSFET

switching mechanism improved battery efficiency by ensuring that the main system remained powered only when the helmet was worn. The use of boost and buck converters provided stable voltage regulation, preventing performance degradation during GSM transmission peaks. Overall, the system demonstrated stable real-time operation, reliable accident detection capability, reduced false positives, and efficient power management, validating its practicality for real-world two-wheeler safety applications.

Verification Mechanism and Communication Performance

The SMS-based accident verification mechanism significantly reduced false alerts. In test cases involving sudden braking and pothole disturbances, the system initially detected abnormal motion but avoided unnecessary emergency notifications after receiving a “NO” response from the rider. In scenarios where no response was provided within the 5-minute window, the system automatically transmitted emergency alerts, demonstrating effective timeout handling. GPS location retrieval showed acceptable accuracy under open-sky conditions, with location coordinates transmitted successfully through the SIM800L GSM module within a few seconds.

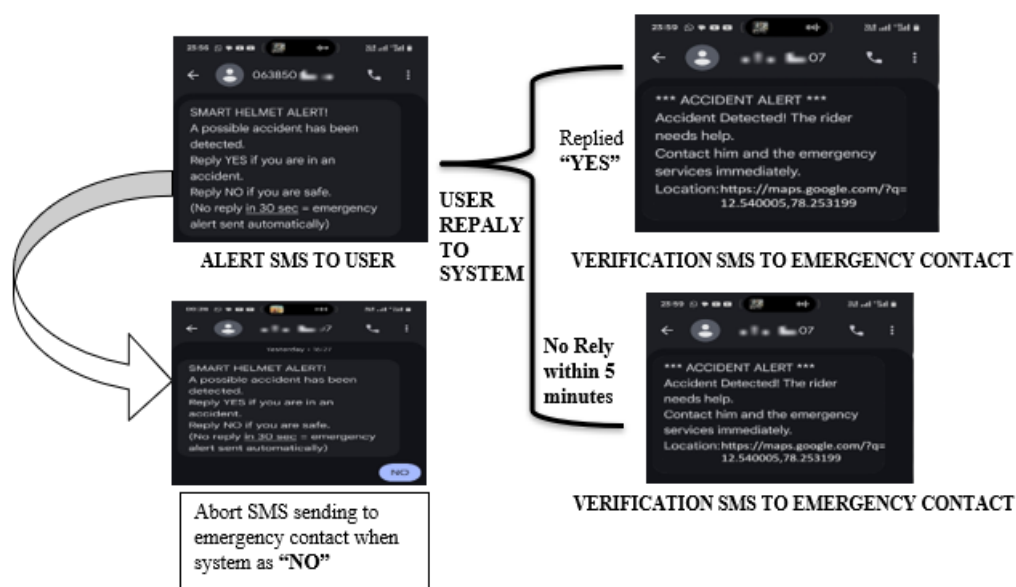


Figure 9.4: SMS ALERT TO USER & EMERGENCY CONTACT

CHAPTER 10

CONCLUSION AND FUTURE ENHANCEMENT

The proposed IoT-Based Smart Helmet system presents an intelligent and integrated approach to enhancing rider safety by combining motion sensing, accident detection, GPS tracking, GSM communication, and helmet-wearing verification into a single wearable platform. Powered by the ESP32 microcontroller, the system ensures real-time monitoring, rapid data processing, and reliable emergency communication. The MPU6050 sensor accurately detects abnormal acceleration and tilt conditions indicative of crash scenarios, while the Hall sensor enforces proper helmet usage through controlled power activation using a PNP transistor-based switching mechanism. A key innovation of the system is the accident verification process, wherein an initial SMS is sent to the rider to confirm the occurrence of an accident before notifying emergency contacts. This intelligent confirmation mechanism significantly reduces false alerts while maintaining reliability. In situations where the rider is unable to respond, the automatic timeout feature ensures emergency alerts are transmitted along with precise GPS location data. Overall, the system offers a cost-effective, scalable, and practical solution designed to minimize accident-related fatalities and improve emergency response efficiency for two-wheeler riders.

Future Enhancement

- ✓ Implementation of **automatic engine cut-off mechanism** during severe crash detection to prevent further damage.
- ✓ Cloud-based storage for **access logs and user management**
- ✓ Development of a dedicated **mobile application** for real-time tracking, alert management, and system status monitoring.
- ✓ Direct **integration with nearby hospitals or emergency services** for faster medical response.

REFERENCES

- [1] S. K. Bhoi and P. M. Khilar, “Vehicular Accident Detection and Reporting System Using Smartphones,” in *Proc. IEEE Int. Conf. Advanced Networks and Telecommunications Systems (ANTS)*, pp. 1–6, 2018.
- [2] D. Joshi and S. Mehta, “Accident Detection System Using Sensors,” *Int. J. Eng. Res. Technol. (IJERT)*, vol. 7, no. 5, pp. 234–238, 2018.
- [3] T. Patel and D. Shah, “Real-Time Accident Detection Using Accelerometer and GSM,” *Int. J. Innovative Technology and Exploring Engineering (IJITEE)*, vol. 8, no. 6, pp. 1200–1204, 2019.
- [4] R. Kumar and S. Raj, “IoT-Based Smart Helmet for Accident Detection,” *IEEE Xplore Digital Library*, pp. 1–5, 2020.
- [5] K. S. Reddy and V. Lakshmi, “Smart Helmet with Alcohol Detection and Accident Notification,” *Int. J. Eng. and Advanced Technology (IJEAT)*, vol. 9, no. 4, pp. 150–155, 2020.
- [6] J. Lee, H. Kim, and S. Park, “Wearable Safety Device Using MEMS Sensors,” *IEEE Sensors Journal*, vol. 20, no. 15, pp. 8500–8507, 2020.
- [7] M. S. Hossain and G. Muhammad, “IoT-Based Intelligent Helmet System for Accident Detection,” *IEEE Internet of Things Journal*, vol. 8, no. 10, pp. 850–858, 2021.
- [8] Y. Zhang, L. Wang, and H. Sun, “MEMS-Based Sensor Systems for Intelligent Transportation,” *Sensors (MDPI)*, vol. 21, no. 4, pp. 1–15, 2021.
- [9] K. Ahmed and M. Rahman, “GSM and GPS-Based Accident Alert System,” *Int. J. Computer Science Engineering*, vol. 9, no. 3, pp. 45–50, 2021.
- [10] M. Khan, A. Ali, and S. Rehman, “IoT-Based Transportation Safety System,” *Int. J. Smart Technology*, vol. 5, no. 2, pp. 60–66, 2021.
- In [11] S. N. Shinde and P. Patil, “Two-Wheeler Accident Detection Using GSM and GPS,” *Int. J. Advanced Research in Engineering and Technology (IJARET)*, vol. 15, no. 2, pp. 89–96, 2024
- In[12] V. Rao and S. Prasad, “Accident Detection System with SMS Verification,” in *Proc. IEEE Conf. Smart Systems*, pp. 210–215, 2024.

ANNEXURE I

CODING

***//DESIGN AND IMPLEMENTATION OF A BUCKLE-ACTIVATED IOT
SMART HELMET FOR AUTONOMOUS ACCIDENT DETECTION AND
EMERGENCY ALERTING//***

```
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <TinyGPS++.h>
#include <HardwareSerial.h>
#include <math.h>

#define GSM_RX_PIN 13
#define GSM_TX_PIN 14

#define GPS_RX_PIN 16
#define GPS_TX_PIN 17

const char* RIDER_NUMBER = "+918428573108".

const char* EMERGENCY_NUMBER_1 = "+918428573108".
const char* EMERGENCY_NUMBER_2 = "+918144063456"; // set "" to disable

#define ACCEL_MAG_THRESHOLD 294.3f // m/s2 = 30g (vibration limit)
#define ACCEL_DELTA_THRESHOLD 49.0f // m/s2 = ~5g (sudden accel change)
#define GYRO_MAG_THRESHOLD 500.0f // deg/s (sudden rotation)

#define REPLY_TIMEOUT_MS 30000UL // 30 seconds reply window
#define GPS_TIMEOUT_MS 20000UL // 20 seconds GPS fix window

Adafruit_MPU6050 mpu;

HardwareSerial gsmSerial(1); // UART1 → SIM800L
HardwareSerial gpsSerial(2); // UART2 → Neo-6M
TinyGPSPlus gps;

enum State {
    STATE_MONITORING,
    STATE_CRASH_DETECTED,
    STATE_AWAITING_REPLY,
    STATE_ALERT_YES, // Rider replied YES → confirmed accident
    STATE_ALERT_NOREPLY, // No reply in 30s → assumed accident
```

```

    STATE_ABORTED      // Rider replied NO → false alarm
};
State currentState = STATE_MONITORING.

float    lastAccelMag  = 9.81f; // ~1g at rest
double   savedLat     = 0.0.
double   savedLng     = 0.0.
bool     gpsFixed     = false.

bool detectCrash();
bool acquireGPS();
String buildLocationStr();
void sendVerificationSMS();
int  waitForReply();
void sendAlertYES();
void sendAlertNoReply();
void sendSMS(const char* number, String message);
String readGSM(unsigned long timeoutMs);
void gsmCommand(const char* cmd, unsigned long waitMs);
void initGSM();
void log(const char* msg);

void setup() {
    Serial.begin(115200);
    delay(200);
    log("=== Smart Helmet Starting ===");

    Wire.begin(21, 22);
    if (! mpu.begin()) {
        log("[MPU6050] NOT FOUND – check wiring!");
        while (1) {delay (500);}
    }

    mpu.setAccelerometerRange(MPU6050_RANGE_16_G);

    mpu.setGyroRange(MPU6050_RANGE_2000_DEG);

    mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);

    log("[MPU6050] OK – Range: ±16g | Gyro: ±2000 deg/s | LPF: 21Hz");

    gsmSerial.begin(9600, SERIAL_8N1, GSM_RX_PIN, GSM_TX_PIN);
    delay (3000);
    initGSM();

    // GPS Neo-6M – UART2

```

```

gpsSerial.begin(9600, SERIAL_8N1, GPS_RX_PIN, GPS_TX_PIN);
log("[GPS] UART started");

log("System ACTIVE – Monitoring started");
}

void loop() {

// Always feed GPS parser
while (gpsSerial.available()) {
    gps.encode(gpsSerial.read());
}

switch (currentState) {

case STATE_MONITORING:
    if (detectCrash()) {
        log(">>> CRASH DETECTED <<<");
        currentState = STATE_CRASH_DETECTED;
    }
    break;

case STATE_CRASH_DETECTED:
    log("Acquiring GPS location...");
    gpsFixed = acquireGPS();
    savedLat = gps.location.lat();
    savedLng = gps.location.lng();

    sendVerificationSMS();
    currentState = STATE_AWAITING_REPLY;
    log("Verification SMS sent. Waiting for rider reply (30s)...");
    break;

case STATE_AWAITING_REPLY: {
    int reply = waitForReply();
    if (reply == 1) currentState = STATE_ALERT_YES;
    else if (reply == 0) currentState = STATE_ABORTED;
    else currentState = STATE_ALERT_NOREPLY;
    break;
}

case STATE_ALERT_YES:
    sendAlertYES();
    log("Alert sent (YES). Resuming monitoring...");
    delay(5000);
    lastAccelMag = 9.81f;

```



```

    currentState = STATE_MONITORING;
    break;

case STATE_ALERT_NOREPLY:
    sendAlertNoReply();
    log("Alert sent (no reply). Resuming monitoring...");
    delay(5000);
    lastAccelMag = 9.81f;
    currentState = STATE_MONITORING;
    break;

case STATE_ABORTED:
    log("Rider replied NO – Rider is safe. No alert sent. Resuming monitoring...");
    lastAccelMag = 9.81f;
    delay(2000);
    currentState = STATE_MONITORING;
    break;
}
}

bool detectCrash() {
    sensors_event_t accelEvent, gyroEvent, tempEvent;
    mpu.getEvent(&accelEvent, &gyroEvent, &tempEvent);

    float ax      = accelEvent.acceleration.x;
    float ay      = accelEvent.acceleration.y;
    float az      = accelEvent.acceleration.z;
    float accelMag = sqrtf(ax*ax + ay*ay + az*az);

    float accelDelta = fabsf(accelMag - lastAccelMag);
    lastAccelMag     = accelMag;

    float gx      = gyroEvent.gyro.x * (180.0f / M_PI);
    float gy      = gyroEvent.gyro.y * (180.0f / M_PI);
    float gz      = gyroEvent.gyro.z * (180.0f / M_PI);
    float gyroMag  = sqrtf(gx*gx + gy*gy + gz*gz);

    bool vibration  = (accelMag >= ACCEL_MAG_THRESHOLD); // 30g impact
    bool suddenAccel = (accelDelta >= ACCEL_DELTA_THRESHOLD); // sudden
    spike
    bool suddenGyro  = (gyroMag >= GYRO_MAG_THRESHOLD); // violent
    rotation

    Serial.printf("[MPU6050] accelMag=%.2f m/s² delta=%.2f m/s² gyro=%.2f
    deg/s\n",
        accelMag, accelDelta, gyroMag);

```

```

Serial.printf("      Vibration:%s SuddenAccel:%s SuddenGyro:%s\n",
    vibration ? "YES" : "no",
    suddenAccel ? "YES" : "no",
    suddenGyro ? "YES" : "no");

// ALL THREE must trigger simultaneously
return (vibration && suddenAccel && suddenGyro);
}

bool acquireGPS() {
    unsigned long start = millis();
    log("[GPS] Waiting for fix...");

    while (millis() - start < GPS_TIMEOUT_MS) {
        while (gpsSerial.available()) {
            gps.encode(gpsSerial.read());
        }
        if (gps.location.isValid() && gps.location.age() < 2000) {
            Serial.printf("[GPS] Fix OK: %.6f, %.6f\n",
                gps.location.lat(), gps.location.lng());
            return true;
        }
        delay(100);
    }

    log("[GPS] Timeout – alert will be sent without accurate location.");
    return false;
}

String buildLocationStr() {
    if (gpsFixed && gps.location.isValid()) {
        String loc = "Location: https://maps.google.com/?q=";
        loc += String(savedLat, 6);
        loc += ",";
        loc += String(savedLng, 6);
        return loc;
    }
    return "Location: GPS unavailable. Please track the rider manually.";
}

void sendVerificationSMS() {
    log("[GSM] Sending verification SMS to rider...");
    String msg =
        "SMART HELMET ALERT!\n"
        "A possible accident has been detected.\n"
        "Reply YES if you are in an accident.\n"

```

```

    "Reply NO if you are safe.\n"
    "(No reply in 30 sec = emergency alert sent automatically)";
    sendSMS(RIDER_NUMBER, msg);
}

int waitForReply() {
    unsigned long deadline = millis() + REPLY_TIMEOUT_MS;

    while (millis() < deadline) {
        // Keep feeding GPS while waiting
        while (gpsSerial.available()) {
            gps.encode(gpsSerial.read());
        }

        String incoming = readGSM(800);

        if (incoming.length() > 0) {
            Serial.print("[GSM] Incoming: "); Serial.println(incoming);
            String upper = incoming;
            upper.toUpperCase();

            if (upper.indexOf("YES") != -1) {
                log("[GSM] Rider replied YES – Accident confirmed.");
                return 1;
            }
            if (upper.indexOf("NO") != -1) {
                log("[GSM] Rider replied NO – Rider is safe.");
                return 0;
            }
        }
    }

    delay(100);
}

log("[GSM] No reply received in 30s.");
return -1;
}

void sendAlertYES() {
    String msg =
        String("*** ACCIDENT ALERT ***\n") +
        "Accident Detected! The rider needs help.\n" +
        "Contact him and the emergency services immediately.\n" +
        buildLocationStr();

    log("[GSM] Sending alert (rider confirmed YES)...");
}

```

```

    if (strlen(EMERGENCY_NUMBER_1) > 0) {
        sendSMS(EMERGENCY_NUMBER_1, msg);
        delay(3000);
    }
    if (strlen(EMERGENCY_NUMBER_2) > 0) {
        sendSMS(EMERGENCY_NUMBER_2, msg);
        delay(3000);
    }

    log("[GSM] Alert sent to all emergency contacts.");
}

void sendAlertNoReply() {
    String msg =
        String("*** ACCIDENT ALERT ***\n") +
        "The rider met with an accident and did not reply to the verification.\n" +
        "Contact the emergency services and him immediately.\n" +
        buildLocationStr();

    log("[GSM] Sending alert (no reply from rider)...");

    if (strlen(EMERGENCY_NUMBER_1) > 0) {
        sendSMS(EMERGENCY_NUMBER_1, msg);
        delay(3000);
    }
    if (strlen(EMERGENCY_NUMBER_2) > 0) {
        sendSMS(EMERGENCY_NUMBER_2, msg);
        delay(3000);
    }

    log("[GSM] Alert sent to all emergency contacts.");
}

void sendSMS(const char* number, String message) {
    Serial.printf("[GSM] SMS to %s\n", number);

    String cmd = "AT+CMGS=\"";
    cmd += number;
    cmd += "\"";
    gsmSerial.println(cmd);
    delay(1000);

    gsmSerial.print(message);
    gsmSerial.write(26); // Ctrl+Z to send
    delay(5000);
}

```

```

String result = readGSM(5000);
Serial.print("[GSM] Send result: "); Serial.println(result);
}

String readGSM(unsigned long timeoutMs) {
    String response = "";
    unsigned long t = millis();
    while (millis() - t < timeoutMs) {
        while (gsmSerial.available()) {
            char c = (char)gsmSerial.read();
            response += c;
        }
    }
    response.trim();
    return response;
}

void gsmCommand(const char* cmd, unsigned long waitMs) {
    gsmSerial.println(cmd);
    delay(waitMs);
    String r = readGSM(waitMs);
    Serial.print("[GSM] "); Serial.print(cmd);
    Serial.print(" → "); Serial.println(r);
}

void initGSM() {
    log("[GSM] Initializing SIM800L...");
    gsmCommand("AT", 1000);
    gsmCommand("ATE0", 500);
    gsmCommand("AT+CMGF=1", 500);
    gsmCommand("AT+CNMI=2,2,0,0,0", 500);
    gsmCommand("AT+CMGDA=\"DEL ALL\"", 3000);
    log("[GSM] SIM800L Ready");
}

void log(const char* msg) {
    Serial.println(msg);
}

```